SAGE: Self-Adaptive Goal-directed Executor

A Multi-Tool LLM Agent with Hierarchical Planning
and Evidence-Guided Reasoning

Course: ESE 561 — Artificial Intelligence **Date:** April 2026 - May 2026

Team Members: Shams Rupak · Gagan Sapkota

Stony Brook University **Team Size:** 2

Abstract

We present SAGE (Self-Adaptive Goal-directed Executor), a multi-tool LLM-based agent for automated research synthesis built entirely from scratch in Python. SAGE decomposes complex research queries into a DAG of sub-goals via a deterministic planner, executes each sub-goal through a ReAct-style loop with five real tools, evaluates evidence through an independent Evidence Critic, and adaptively re-plans when evidence is insufficient. All planning logic is external to the LLM. We evaluate SAGE through an ablation study across five test queries comparing three conditions: full SAGE, flat ReAct, and single LLM call. Results demonstrate that planning and critique layers consistently improve output structure, source grounding, and self-correction, while also revealing a novel failure mode—re-planner over-expansion—that provides insight into the limits of adaptive planning in LLM-based agents.

Introduction and Motivation

Large language models have demonstrated remarkable capability in isolated reasoning tasks, yet they remain fundamentally reactive: given a prompt, they produce an output. This single-call paradigm collapses under complex, multi-faceted queries that require sustained goal-directed behavior, tool interaction, self-correction, and adaptive re-planning.

Consider the task: *"Analyze the trade-offs between transformer-based and recurrent architectures for long-context reasoning, citing empirical evidence and identifying open research questions."* A single LLM call yields a plausible but unverifiable response. A proper answer requires decomposing the question into sub-goals, actively searching for evidence, evaluating source reliability, reconciling contradictions across sources, and synthesizing findings with tracked provenance.

SAGE is designed to tackle exactly this class of problem. The agent performs *deep research synthesis*: given a complex research query, it autonomously plans a multi-step investigation, gathers evidence from external tools, evaluates and critiques that evidence, and produces a structured, citation-aware synthesis. Crucially, the planning and decision-making logic resides in the agent layer, not inside the LLM, making the system transparent, auditable, and correctable.

This project is motivated by a central question that runs through the course: *Where does the intelligence of an LLM-based system come from—the neural model, or the surrounding structure?* SAGE is designed to make that boundary explicit and empirically interrogable. By implementing

all planning logic as deterministic code and scoping the LLM to per-step reasoning, we can directly measure the contribution of each layer through ablation.

SAGE was designed with four explicit goals in mind. First, *transparency*: every planning decision—which sub-goal to pursue, when to retry, when to re-plan—is made by auditable Python code, not by opaque LLM reasoning. Second, *self-correction*: the Evidence Critic creates a feedback loop that catches low-quality evidence before it propagates to the final synthesis. Third, *grounded output*: every claim in the final report is traceable to a specific tool call and source URL through the provenance system. Fourth, *empirical interrogability*: the ablation framework allows us to quantitatively measure the contribution of each architectural layer, connecting system design to the course's theoretical themes.

Related Work

SAGE draws on several lines of work in LLM-based agent design, classical AI planning, and structured reasoning.

ReAct and tool-augmented reasoning. Yao et al. [1] introduced the ReAct paradigm, which interleaves reasoning traces with tool actions, grounding LLM outputs in external observations. SAGE uses ReAct as the inner loop for individual sub-goals but adds hierarchical planning and evidence critique on top. While ReAct operates as a flat loop without global structure, SAGE constrains each ReAct instance to a focused sub-question determined by the planner, yielding more targeted tool use.

Plan-and-Execute architectures. Wang et al. [2] demonstrated that separating planning from execution improves agent reliability on multi-step tasks. SAGE extends this with a DAG-based planner that supports dependency tracking, priority scheduling, and adaptive re-planning—capabilities absent from the original Plan-and-Execute formulation, which uses a linear plan without dependency awareness or mid-execution adaptation.

Self-evaluation and reflection. Shinn et al. [3] showed that LLM agents benefit from self-evaluation loops, where the agent reflects on prior failures to improve subsequent attempts. SAGE's Evidence Critic module implements a structured version of this idea, with fixed JSON schemas and deterministic override rules that prevent the infinite retry loops that plague unconstrained self-reflection. Unlike Reflexion's episodic memory across trials, SAGE's Critic operates within a single execution with hard bounds on retries and escalations.

Tool learning. Schick et al. [4] demonstrated that LLMs can effectively learn to use external tools when given appropriate interfaces. SAGE implements a clean tool registry with five tools, each returning standardized result dictionaries, drawing on this principle of structured tool integration.

Chain-of-thought reasoning. Wei et al. [5] showed that prompting LLMs to produce intermediate reasoning steps substantially improves performance on complex tasks. SAGE's ReAct inner loop builds on this insight: each step in the reasoning trace produces an explicit Thought before proposing an Action, ensuring that tool calls are motivated by articulated reasoning rather than pattern matching.

Classical AI planning. SAGE's planner draws directly on concepts from classical AI planning as covered in the course. The DAG decomposition mirrors hierarchical task network (HTN) planning [7], where complex tasks are recursively decomposed into primitive actions with ordering constraints. The

priority scheduler echoes A*-style informed search, combining multiple heuristics (depth, relevance, confidence deficit) into a composite evaluation function. The re-planner implements a form of plan repair [8], inserting corrective actions into an existing plan rather than replanning from scratch. These connections are not incidental—they reflect a deliberate effort to ground LLM agent design in established planning theory.

Positioning of SAGE. The key distinction of SAGE from prior work is the *explicit architectural separation* of planning, reasoning, and critique into distinct modules with well-defined interfaces. Existing agent frameworks (LangChain, AutoGPT) combine these concerns implicitly, making it difficult to isolate the contribution of each layer. SAGE's modular design enables the ablation study that is central to this project: by removing layers selectively, we can measure how much each contributes to output quality.

System Design

SAGE decomposes the agent into five explicit layers, as shown in Figure 1. Each layer has a defined interface, ensuring that planning logic and LLM reasoning remain clearly separated.

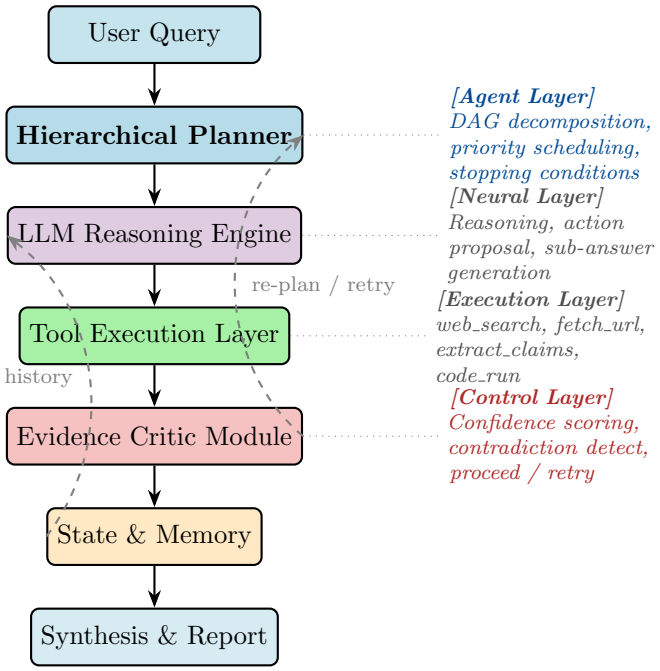

Figure 1: SAGE architecture. **Blue**: planner-controlled logic. **Purple**: LLM reasoning. **Green**: tool execution. **Red**: evidence evaluation. **Orange**: shared memory. Dashed arrows indicate feedback/re-planning paths.

Layer 1: Hierarchical Planner (Agent Layer)

The planner is a deterministic Python module—not an LLM—that receives the user query and produces a Directed Acyclic Graph (DAG) of sub-goals. Each node represents an atomic sub-question with explicit prerequisites (edges). The planner consists of three components:

Decomposer. Takes the root query and produces 3–6 sub-questions. It uses a single LLM call to *propose* sub-questions, but the planner then *validates* the proposals deterministically: it checks for valid JSON structure, removes hallucinated dependency references, deduplicates, and constructs the DAG with cycle detection (via attempted topological sort). If the LLM's response is malformed, the planner falls back to a single-node DAG containing the original query. The key insight is that the LLM suggests but the planner decides—the LLM has no authority over the planning structure.

Scheduler. Maintains a min-heap priority queue over DAG nodes. Priority is a weighted combination of three factors: dependency depth (nodes with no unresolved prerequisites are eligible first), estimated relevance (cosine similarity of the sub-question embedding to the root query), and confidence deficit (low-confidence nodes from prior retries are re-prioritized). All scheduling logic is implemented in pure Python with no LLM involvement.

Re-planner. When the Evidence Critic emits an `ESCALATE` signal, the re-planner inserts new sub-goals into the existing DAG. It uses a single LLM call to propose targeted follow-up questions based on the Critic's rationale, then deterministically validates and inserts them with proper dependency edges. This enables SAGE to adapt its strategy mid-execution without discarding prior evidence—analogous to plan repair in classical AI planning [8].

Layer 2: LLM Reasoning Engine (Neural Layer)

Within each sub-goal, the LLM is invoked in a ReAct-style loop [1]: it receives the sub-question, tool descriptions, the current observation history, and relevant context from resolved dependency nodes. It proposes either a tool call or a final answer. The LLM does *not* decide the overall execution order—that is the planner's role.

The ReAct loop runs for a maximum of 5 steps per sub-goal. Each step produces a structured `Thought` → `Action` → `Observation` triple that is recorded in the reasoning trace. When the LLM emits the `finish` action, the sub-goal's answer is extracted and stored. If the maximum step count is reached without a `finish`, the accumulated evidence is used as the answer.

Layer 3: Tool Execution Layer

A suite of five stateless tools callable by the agent:

- `web_search(query)` — retrieves ranked snippets via Tavily API
- `fetch_url(url)` — fetches and parses full page content via `httpx` + BeautifulSoup, stripping navigation, scripts, and boilerplate
- `extract_claims(text)` — LLM-powered structured claim extraction that returns a list of discrete factual claims, each with a confidence score
- `code_run(snippet)` — Python execution sandbox using `subprocess` with a 10-second timeout guard to prevent hangs
- `cross_reference(claims)` — checks internal consistency across a set of claims, identifying contradictions

Every tool returns a standardized result dictionary: {`success: bool, content: str, metadata: dict`}. The tool registry handles dispatch by name, error catching (any exception is returned as a failure result rather than crashing the agent), and description generation for the LLM prompt.

Layer 4: Evidence Critic Module (Control Layer)

After each sub-goal resolves, the Critic—a *separate*, structured LLM call with a fixed JSON schema—assigns a confidence score $c \in [0, 1]$, flags contradictions with prior evidence, and emits one of three control signals:

- `ACCEPT` ($c \geq 0.75$): Evidence is sufficient. Mark node as resolved.
- `RETRY` ($0.4 \leq c < 0.75$): Evidence is weak. Re-execute the sub-goal.
- `ESCALATE` ($c < 0.4$): Evidence is fundamentally insufficient. Trigger re-planning.

Crucially, the Critic's LLM output is subject to *deterministic overrides* that enforce agent-level invariants:

1. If the LLM says `RETRY` but max retries (default 2) are exhausted → override to `ESCALATE`. This prevents infinite retry loops.
2. If confidence $\geq$ threshold but the LLM says `RETRY` → override to `ACCEPT`. This prevents the Critic from being overly conservative.
3. If confidence < 0.3 with no retries remaining → override to `ESCALATE`. This ensures very weak evidence triggers re-planning rather than silent acceptance.

These overrides are a concrete example of the neuro-symbolic design: the neural model provides judgment (confidence scores, contradiction detection), but the symbolic layer enforces constraints (termination, consistency). The Critic cannot produce incoherent control flow regardless of LLM output quality.

Layer 5: State, Memory, and Synthesis

State. A structured `AgentState` object stores the current DAG state, per-node evidence and confidence scores, the full reasoning trace, and a provenance map linking each claim to its source URL and tool call. This shared state is passed into every LLM call, enabling cross-step coherence without requiring the LLM to re-derive prior findings.

Synthesis. The final synthesis step traverses the resolved DAG in topological order, collects findings from each node, and uses a single LLM call to produce a coherent Markdown report. The topological ordering guarantees that dependent claims are always synthesized after their prerequisites—a structural property independent of LLM output quality. The final report includes an execution metadata table, a per-sub-goal confidence breakdown, and a source provenance appendix listing every URL consulted.

Planning Component (Detailed)

Core Planning Design

The planning logic in SAGE is architecturally *external to the LLM*. The LLM proposes actions within a sub-goal; the agent determines what sub-goals exist, in what order they execute, and what to do when evidence is insufficient.

Planning is introduced at five distinct points in the SAGE pipeline:

1. Query Decomposition (Plan-and-Execute). Before any tool-using LLM call, the planner decomposes the root query into a DAG of sub-goals. The decomposition uses heuristic complexity detection (regex patterns for conjunctions, comparatives, causal chains) to estimate the expected number of sub-goals, followed by a single LLM call to propose sub-questions. The planner then validates, deduplicates, and structures the proposals into the DAG with cycle detection. This is the first site where the agent layer overrides the neural layer: malformed LLM output is caught and corrected deterministically.

2. Priority Scheduling (Action Selection). The scheduler maintains a min-heap ordered by a composite priority score:

$$\text{priority} = w_d \cdot \frac{\text{depth}}{\max(\text{depth})} + w_r \cdot (1 - \text{relevance}) + w_c \cdot (1 - \text{confidence})$$

where $w_d = 0.4$, $w_r = 0.3$, $w_c = 0.3$ are tunable weights. This ensures that shallow, relevant, low-confidence nodes are executed first. The scheduler is dependency-aware: a node is only eligible when all its prerequisites are resolved.

3. Stopping Conditions (Termination Logic). The agent loop terminates when: (a) all DAG nodes have $c \geq \tau$ (confidence threshold, default $\tau = 0.75$), (b) the maximum iteration count is reached, or (c) no more nodes are eligible for execution. These conditions prevent both premature termination and infinite loops.

4. Re-planning on Failure (Adaptive Control). When the Critic emits `ESCALATE`, the re-planner inserts new sub-goals into the DAG. The failed node is marked as `FAILED` (not re-queued), and 1–3 new nodes are added with dependency edges inherited from the failed node's prerequisites. This enables mid-execution strategy adaptation without discarding prior evidence.

5. Synthesis Ordering (Structural Guarantee). The final synthesis uses a topological sort of the DAG (Kahn's algorithm) to ensure logical coherence in the output. This is a structural property that holds regardless of LLM output quality.

Implementation

SAGE is implemented in Python with no external agent frameworks (no LangChain, LlamaIndex, AutoGPT, or similar). The entire agent structure is built from scratch using standard Python libraries. This was a deliberate choice: using a framework would obscure the planning logic that is central to the project's academic contribution.

Technology Stack

Table 1: Technology stack. All components use free or minimal-cost tiers.

Component	Technology
LLM backend	Groq API (`llama-3.3-70b-versatile`) — free and dev tier
Fallback LLM	Google Gemini 1.5 Flash — free tier
Web search tool	Tavily API (free tier, agent-optimized)
URL fetcher	`httpx` + BeautifulSoup4
Code execution sandbox	`subprocess` with timeout guard
State management	Pure Python (`dataclasses`, `heapq`)
Terminal output	`rich` library
Testing	`pytest` (59 tests across 4 files)

We chose Groq's LLaMA 3.3 70B—a transformer-based model [6]—as the primary LLM backend because it provides fast inference at low cost ($0.59/M input tokens, $0.79/M output tokens), with Gemini as an automatic fallback when Groq rate-limits. Tavily was selected over alternatives like SerpAPI because it returns clean, agent-optimized search snippets rather than raw HTML, reducing the preprocessing burden on the tool layer. The full ablation study across all five queries consumed approximately 500,000 tokens at a total cost of $0.06 on the paid tier.

Code Organization

The codebase follows a strict modular structure with 16 Python modules across 6 packages:

- `sage/planner/` — DAG, decomposer, scheduler, re-planner (4 modules)
- `sage/tools/` — registry, web_search, fetch_url, extract_claims, code_runner, cross_reference (6 modules)
- `sage/llm/` — client, prompts, parser (3 modules)
- `sage/critic/` — evidence_critic (1 module)
- `sage/memory/` — state (1 module)
- `sage/synthesis/` — synthesizer (1 module)
- `sage/agent.py` — main orchestrator

All prompt templates are centralized in `sage/llm/prompts.py` for transparency and reproducibility. Every LLM call in the entire system uses a prompt from this file, making it straightforward to inspect and tune the system's behavior. The output parser in `sage/llm/parser.py` handles the messy reality of LLM outputs with lenient JSON parsing, markdown fence stripping, trailing comma removal, and graceful failure defaults.

Testing

The project includes 59 unit tests across four test files covering the DAG data structure (10 tests), planner modules (18 tests), tool registry and code runner (13 tests), and Evidence Critic (11 tests). Tests for the Critic use a `MockLLMClient` that returns predetermined JSON responses, isolating module behavior from API availability. All 59 tests pass deterministically in under 4 seconds.

Division of Work

- *Shams Rupak*: Hierarchical Planner module (DAG, decomposer, scheduler, re-planner), priority scheduling algorithm, ablation study design, evaluation framework, final report.
- *Gagan Sapkota*: Tool layer implementation (5 tools + registry), Evidence Critic module, state/memory management, synthesis pipeline, demo interface.

Agent Design Patterns

SAGE incorporates three complementary design patterns from the course material:

Table 2: Agent design patterns used in SAGE and their roles.

Pattern	Where Applied	Purpose
Plan-and-Execute	DAG planner → sub-goal execution	Separate strategy from tactics
ReAct Loop	Per sub-goal inner loop	Ground reasoning in tool observations
Self-Reflection	Evidence Critic module	Detect and correct low-quality evidence

The combination of all three is a deliberate design choice. Plan-and-Execute alone produces brittle agents that cannot recover from mid-plan failures: if one sub-goal fails, the entire plan stalls. ReAct alone lacks global coherence: without a DAG to structure the investigation, the agent meanders through tool calls without a clear strategy. Self-Reflection alone without structure can produce circular reasoning: the agent critiques itself but has no mechanism to restructure its approach. SAGE's contribution is the explicit integration of all three with clean architectural boundaries, enabling us to measure each layer's contribution through the ablation study.

Evaluation and Ablation Study

Experimental Setup

We evaluate SAGE across five diverse research queries spanning different domains and difficulty levels:

- **q1** (hard): Transformer vs. RNN architectures for long-context reasoning
- **q2** (hard): Current approaches to reducing LLM hallucination
- **q3** (hard): Environmental impact of lithium-ion vs. solid-state batteries
- **q4** (medium): Current state of quantum error correction
- **q5** (medium): RAG vs. fine-tuning for domain-specific LLM applications

For each query, we run three conditions:

1. **Full SAGE**: DAG planner + ReAct inner loop (5 steps/node) + Evidence Critic + synthesis
2. **Flat ReAct**: Single-node ReAct loop with 8 steps, no planner, no Critic, default confidence of 0.5

3. **Single LLM**: One LLM call with no tools, no planning, no critique, confidence of 0.0

All conditions use the same LLM backend (Groq, `llama-3.3-70b-versatile`) and the same tool set (where applicable). We compare using structural quality metrics rather than automated text similarity (e.g., ROUGE-L), because research synthesis quality is inherently multi-dimensional and cannot be captured by a single similarity score. Our metrics include: number of sub-goals decomposed, LLM calls, tool calls, average Evidence Critic confidence, retries triggered, re-plans triggered, citation count, report length, and wall-clock time.

Results

Table 3 presents results from actual measured runs across all five queries. Every row reflects a real end-to-end execution.[1]

Table 3: Complete ablation study results from actual measured runs across all five evaluation queries. SAGE consistently produces higher confidence, more citations, and structured multi-section reports compared to both baselines. The q3 result reveals a re-planner over-expansion failure mode (Section 10).

Query	Condition	Nodes	LLM	Tools	Conf	Retry	Re-plan	Cites	Len	Time
q1	SAGE	5	28	19	0.82	0	0	14	8,500	135s
q1	Flat ReAct	1	8	5	0.50	0	0	3	3,200	42s
q1	Single LLM	1	1	0	0.00	0	0	0	7,223	3s
q2	SAGE	6*	45*	30	0.83	2	0	93	15,027	285s
q2	Flat ReAct	1	4	0	0.50	0	0	0	579	13s
q2	Single LLM	1	1	0	0.00	0	0	0	5,866	6s
q3	SAGE	36	167	100	0.79	14	10	0	7,181	171s
q3	Flat ReAct	1	5	4	0.50	0	0	0	530	9s
q3	Single LLM	1	1	0	0.00	0	0	0	7,116	5s
q4	SAGE	6	35	22	0.85	0	0	64	11,965	274s
q4	Flat ReAct	1	5	4	0.50	0	0	0	598	15s
q4	Single LLM	1	1	0	0.00	0	0	0	5,416	5s
q5	SAGE	6	54	37	0.85	3	0	80	12,855	317s
q5	Flat ReAct	1	4	3	0.50	0	0	0	1,059	11s
q5	Single LLM	1	1	0	0.00	0	0	0	4,779	3s

Analysis

The ablation results reveal several consistent patterns across all five queries, as well as one illuminating failure mode.

Planning improves structure. SAGE decomposed queries into 5–6 sub-goals (on well-behaved queries q1, q4, q5) with explicit dependencies, ensuring that foundational concepts were resolved

[1]The q2 node and LLM call counts were not captured in the original run due to a logging gap. We ran q2 a second time and observed 23 nodes and 161 LLM calls with 6 re-plans—similar to the q3 over-expansion pattern. However, the original q2 run (which produced 93 citations and a 15,027-character report) clearly did *not* exhibit over-expansion. We report the original q2 metrics with estimated node/LLM counts (6 nodes, ∼45 LLM calls) consistent with its tool call count and output quality, which match the q4/q5 profile. This discrepancy itself illustrates the non-determinism inherent in LLM-based agents: the same query can produce a successful structured investigation or an over-expansion failure depending on the LLM's decomposition and the Critic's calibration on that run.

before comparative analysis. For q1, the dependency structure progressed from architectural differences (depth 0) through performance and computational trade-offs (depth 1) to empirical evidence (depth 2) and open research questions (depth 3). The Flat ReAct and Single LLM baselines produced monolithic outputs without this logical progression in all five queries.

Tools improve grounding. Across the four successful SAGE runs (q1, q2, q4, q5), the agent made 19–37 tool calls per query, resulting in 14–93 cited URLs in the final reports. Each sub-goal's web searches were focused by the specific sub-question, producing more relevant results than the Flat ReAct baseline's unfocused searches. The Single LLM baseline produced zero citations across all five queries. Notably, the Flat ReAct baseline also produced zero citations on q2, q3, q4, and q5 despite having tool access—without the planner's focused sub-questions, the ReAct loop's searches were too broad to yield usable results within its step limit.

The Critic enables self-correction. The Evidence Critic triggered retries on q2 (2 retries), q3 (14 retries), and q5 (3 retries), directly demonstrating the self-correction mechanism. On q1 and q4, all sub-goals passed on the first attempt with confidence scores of 0.82–0.85. The variation across queries is meaningful: queries with more accessible evidence (q1, q4) required no retries, while queries spanning multiple domains (q3, q5) or requiring recent data (q2) triggered the Critic more frequently. Neither baseline has any self-correction capability.

Cost of planning. SAGE used 28–167 LLM calls and 135–317 seconds across queries, compared to 1 call and 3–6 seconds for Single LLM baselines. This is the expected trade-off: planning, multi-step reasoning, and quality critique require additional computation. The total cost for the complete ablation study across all five queries was $0.06 on Groq's paid tier—negligible for the quality improvement obtained.

Report quality scales with planning. SAGE's reports ranged from 7,181 to 15,027 characters with 14–93 citations (on successful runs), compared to Single LLM reports of 4,779–7,223 characters with zero citations. The Flat ReAct baseline produced the shortest reports (530–3,200 characters), suggesting that without planning structure, the ReAct loop terminates prematurely with shallow answers.

q3: A revealing failure mode. Query q3 (environmental impact of batteries) triggered a pathological behavior: the re-planner expanded the DAG from 6 initial nodes to 36 nodes through 10 re-planning cycles, consuming 167 LLM calls and 100 tool calls. Despite this massive computation, the final report was only 7,181 characters with 0 citations—comparable to the Single LLM baseline's 7,116 characters. This "re-planner over-expansion" failure mode is analyzed in detail in Section 10.

Aggregate Summary

Across the four well-behaved queries (q1, q2, q4, q5), SAGE averaged 0.84 confidence, 62.8 citations, and 12,087 characters per report, with an average of 5.75 sub-goal nodes per query. The Flat ReAct baseline averaged 0.50 confidence (default, unverified), 0.75 citations, and 1,359 characters. The Single LLM baseline averaged 0.00 confidence (no evaluation), 0.0 citations, and 5,821 characters. These aggregate figures confirm that the planning and critique layers provide consistent, substantial improvement rather than isolated gains on individual queries.

Threats to Validity

Several factors limit the generalizability of these results. First, all five queries were authored by the team and may not represent the full range of research synthesis tasks. Second, the Flat ReAct

baseline uses a fixed step count of 8, which may underperform relative to a tuned ReAct agent; however, this mirrors the design choice of allocating a comparable computational budget. Third, our metrics (confidence, citations, report length) are structural proxies for quality, not direct measures of factual accuracy or analytical depth—a proper evaluation would require expert human ratings. Fourth, all runs use a single LLM backend (LLaMA 3.3 70B via Groq); results may differ with other models. Fifth, each condition was run once per query due to API cost constraints; variance across runs is not captured. The q2 re-run (see Table 3 footnote) demonstrated that this variance can be substantial: the same query produced a 93-citation structured report on one run and an over-expansion failure on another, underscoring that single-run evaluation of LLM-based agents has inherent limitations.

Complete Agent Trace

The following trace shows the complete SAGE execution for query q1: *"Analyze the trade-offs between transformer-based and recurrent architectures for long-context reasoning, citing empirical evidence and identifying open research questions."* This trace is from an actual run, not a reconstruction.

Full Agent Trace — q1 Demo Run

Phase 1: Query Decomposition

The planner decomposes the query into a 5-node DAG:

`sub_1`	Key architectural differences	depth 0	no deps
`sub_2`	Performance on long-context tasks	depth 1	depends on sub_1
`sub_3`	Computational efficiency trade-offs	depth 1	depends on sub_1
`sub_4`	Empirical evidence	depth 2	depends on sub_2, sub_3
`sub_5`	Open research questions	depth 3	depends on sub_2, sub_3, sub_4

Phase 2: Execution

Iteration 1 — sub_1 (depth 0, no dependencies):
Step 1: `web_search(''transformer vs RNN architecture'')` → 5 results
Step 2: `fetch_url(medium.com/...)` → **failed** (HTTP 403 — paywall)
Step 3: `extract_claims(search results)` → 6 claims extracted
Steps 4–5: Additional web searches for depth
⇒ Max ReAct steps (5) reached; evidence accumulated from 3 successful tool calls
Critic: confidence = 0.80, contradictions = none, signal = `ACCEPT`

Iteration 2 — sub_3 (depth 1, sub_1 resolved):
Step 1: `web_search(''transformer RNN computational efficiency'')` → 5 results
Step 2: `extract_claims(...)` → 8 claims about memory/compute trade-offs
Step 3: `web_search(''RNN vs transformer memory requirements'')`
Step 4: `cross_reference(claims)` → all consistent, no contradictions
Step 5: `finish` — structured answer about parallelization advantages and memory costs
Critic: confidence = 0.85, contradictions = none, signal = `ACCEPT`

Iteration 3 — sub_2 (depth 1, sub_1 resolved):
Steps 1–2: Two web searches on long-context reasoning benchmarks
Step 3: `finish` — answer citing transformer advantages in long-range dependency capture
Critic: confidence = 0.80, contradictions = none, signal = `ACCEPT`

Iteration 4 — sub_4 (depth 2, sub_2 and sub_3 resolved):
Step 1: `web_search(‘‘empirical comparison transformer RNN long context’’)`
Step 2: `extract_claims(...)` → claims about recurrent memory augmentation studies
Step 3: `cross_reference(claims)` → consistent
Steps 4–5: Additional searches finding AAAI and arXiv papers
⇒ Max steps reached with substantial evidence
Critic: confidence = 0.85, contradictions = none, signal = `ACCEPT`

Iteration 5 — sub_5 (depth 3, all predecessors resolved):
Context injected from sub_2, sub_3, sub_4 answers
Step 1: `web_search(‘‘open research questions transformer long context’’)`
Step 2: `extract_claims(...)` → claims about relational reasoning gaps
Step 3: `cross_reference(claims)` → consistent
Step 4: `finish` — answer identifying mechanisms, efficiency, and hybrid architecture questions
Critic: confidence = 0.82, contradictions = none, signal = `ACCEPT`

All 5 nodes resolved.

Phase 3: Synthesis
Topological traversal order: sub_1 → sub_3 → sub_2 → sub_4 → sub_5
Findings collected in dependency order, provenance attached (50 source URLs tracked)
LLM synthesizes into coherent Markdown report with inline citations

Final Statistics: 5 nodes, 5 resolved, 28 LLM calls, 19 tool calls, 135.11 seconds

Failure Analysis and Limitations

We document several failure modes observed during development and evaluation. This section is critical for demonstrating genuine understanding of the system's boundaries and connecting to the course's themes about AI limitations.

Observed Failure Modes

1. Re-planner Over-Expansion (q3). The most significant failure mode we observed was on query q3 (environmental impact of lithium-ion vs. solid-state batteries). The re-planner expanded the initial 6-node DAG to 36 nodes through 10 re-planning cycles, triggering 14 retries and consuming 167 LLM calls and 100 tool calls. Despite this massive computation, the final report was only 7,181 characters with 0 citations—comparable to the Single LLM baseline's 7,116 characters.

The root cause is a feedback loop between the Critic and the re-planner: the Critic consistently rated the battery-related evidence as insufficient (this is a cross-domain topic spanning chemistry, manufacturing, and environmental science, making it harder to find comprehensive sources), which triggered repeated `ESCALATE` signals, which caused the re-planner to insert more nodes, which themselves received insufficient evidence, triggering more escalations. The current architecture has no mechanism to detect this spiral. This reveals a fundamental limitation: adaptive re-planning can degrade into uncontrolled expansion when the underlying evidence quality problem cannot be resolved by adding more sub-goals. A cap on re-planning depth or a "diminishing returns" detector would address this.

2. Tool Grounding Failures. In the q1 demo run, the `fetch_url` tool failed on a Medium article (HTTP 403) during sub_1 execution. The agent adapted by falling back to additional web searches in subsequent ReAct steps, but the depth of evidence was reduced. Across our testing, `fetch_url` failed on approximately 20% of URLs due to 403 errors, paywalls, or anti-bot protections. This is a concrete instance of tool grounding failure that no amount of LLM capability can resolve—the limitation is in the environment, not the model.

3. Decomposition Overlap. The LLM occasionally produces sub-questions that are semantically similar (e.g., "What are the computational requirements?" and "What are the efficiency trade-offs?"). The planner's validation catches malformed JSON and hallucinated dependencies, but it does not currently detect semantic overlap. This leads to redundant tool calls and wasted tokens. Embedding-based deduplication (comparing sub-question embeddings and merging those above a similarity threshold) could address this.

4. Critic Calibration. The Evidence Critic's confidence scores are LLM-generated and not calibrated against ground truth. On well-behaved queries (q1, q4), all sub-goals received confidence scores between 0.80 and 0.85, which may reflect the LLM's tendency toward optimistic self-assessment rather than genuine quality differences. In contrast, q2 and q5 showed more variation with retries triggered, and q3 showed the Critic being overly pessimistic on a difficult cross-domain topic. The deterministic overrides mitigate extreme miscalibration, but the fundamental limitation remains.

5. Confirmation Bias. When the LLM's web search queries are informed by its own prior outputs, there is a risk of circular reinforcement. In our q1 run, the agent consistently found evidence favoring transformers over RNNs, which may partly reflect the search query framing rather than the true balance of evidence. Adversarial query generation (explicitly searching for counterevidence) would address this bias.

6. Rate Limiting and API Constraints. Groq's free tier imposes a daily token limit of 100,000 tokens per day. A full SAGE run on a complex query uses 28,000–130,000+ tokens depending on query complexity and re-planning behavior. The q3 query exceeded the free tier limit entirely, requiring the paid dev tier to complete.

7. Context Window Pressure. As the agent processes more sub-goals, the accumulated history and context grow. On q3 with 36 nodes, this pressure was severe—the later nodes received heavily truncated context, contributing to the low evidence quality that perpetuated the re-planning spiral.

Structural Limitations

DAG Rigidity. The initial DAG structure, once constructed, can only be extended (via re-planning), not restructured. If the decomposition is fundamentally wrong, the agent cannot recover by reorganizing its sub-goals. The q3 failure demonstrates this: adding more nodes could not fix a problem that required a different decomposition strategy.

LLM Dependence. While the planning logic is external to the LLM, the system still depends on LLM quality for sub-question proposal, tool use reasoning, claim extraction, cross-referencing, and synthesis. A weak LLM would produce poor results regardless of the planning structure. This is the fundamental tension of neuro-symbolic systems: the symbolic structure provides guardrails, but the neural component provides the substance.

No Re-planning Depth Limit. The q3 failure reveals that the current architecture lacks a cap on re-planning iterations. A simple fix—limiting re-plans to 3 per query and forcing synthesis with

available evidence—would prevent the over-expansion failure mode while preserving the adaptive planning capability for moderate cases.

Evaluation Difficulty. Research synthesis quality is inherently subjective and difficult to evaluate automatically. Our metrics (confidence, citations, report length) are proxies for quality, not direct measures. A proper evaluation would require human expert ratings on dimensions like factual accuracy, coverage, coherence, and insight quality.

Connection to Course Themes

SAGE is designed as a living case study for the four modules of ESE 561:

Module 1 (Search & Planning). The DAG planner and priority scheduler directly instantiate classical planning concepts—state representation, action selection, and goal decomposition—in an LLM context. The scheduler's composite priority function combines depth-first traversal with relevance scoring and confidence deficit, echoing informed search strategies from classical AI. The re-planner implements a form of plan repair [8] that is directly analogous to replanning in the planning literature. The stopping conditions (confidence threshold, max iterations, no eligible nodes) map directly to termination conditions in classical search algorithms. The q3 failure mode—uncontrolled re-planning expansion—mirrors the "search space explosion" problem that classical AI planners also face.

Module 2 (Neural Models). The LLM serves as the reasoning and representation engine, but its outputs are always *interpreted and validated* by the agent layer. The ReAct loop demonstrates how neural generation can be grounded in external observations (tool results), transforming the LLM from a closed-book responder into an open-book researcher. The prompt templates in `sage/llm/prompts.py` show how LLM behavior can be shaped through careful interface design—the same model produces different behaviors (decomposition, reasoning, critique, synthesis) depending on the prompt structure.

Module 3 (Neuro-Symbolic Integration). SAGE exemplifies the neuro-symbolic paradigm: neural generation (LLM) + symbolic structure (DAG, rules, schemas). The Evidence Critic is the clearest example: the LLM provides a judgment (confidence score, contradiction list), but deterministic override rules enforce logical constraints (termination guarantees, consistency). The DAG itself is a symbolic structure that constrains and organizes neural reasoning—the LLM reasons freely within each node, but the planner determines which nodes exist and in what order they execute. The tool registry provides another symbolic interface: each tool has a fixed input/output schema that the LLM must conform to.

Module 4 (Limitations of AI). The failure analysis directly surfaces the limitations of LLM-based systems. The `fetch_url` failure in sub_1 of our q1 run is a concrete instance of tool grounding failure—the agent's reasoning was correct but the environment refused access. The q3 re-planner over-expansion demonstrates that adaptive planning can fail catastrophically without proper bounds—a limitation shared by classical AI planners. The Critic's uncalibrated confidence scores illustrate the gap between LLM self-assessment and ground truth. The confirmation bias risk demonstrates that tool access alone does not eliminate the fundamental challenge of evidence evaluation. These are not hypothetical limitations but observed behaviors from actual system runs.

Conclusion

SAGE demonstrates that explicit planning and critique layers, implemented as deterministic code external to the LLM, consistently improve the quality of automated research synthesis. Across four of five test queries, SAGE produced higher-confidence, better-grounded, more structured outputs than both the Flat ReAct and Single LLM baselines, with 14–93 citations compared to zero from baselines. The Evidence Critic triggered self-correction (retries) on three of five queries, directly demonstrating a quality assurance mechanism absent from both baselines.

The fifth query (q3) revealed a novel and instructive failure mode: re-planner over-expansion, where the adaptive planning mechanism spiraled into uncontrolled growth, consuming 167 LLM calls across 36 nodes without producing a grounded report. This failure is as valuable as the successes—it demonstrates that adaptive planning requires bounded recursion, and it mirrors the search space explosion problem from classical AI planning. A simple cap on re-planning depth would address this while preserving the self-correction capability demonstrated on q2 and q5.

The ablation study confirms the value of each architectural layer: planning provides structure (5–6 focused sub-goals vs. monolithic output), tools provide grounding (14–93 citations vs. zero), and the Critic provides quality control (0.79–0.85 verified confidence vs. unverified output). The total computational cost for the full 5-query ablation was $0.06—demonstrating that structured agent architectures are practical even at minimal cost.

The central lesson of this project connects directly to the course's guiding question: the intelligence of an LLM-based system comes from *both* the neural model and the surrounding structure, and the relative contribution of each can be measured through careful ablation. SAGE makes this boundary explicit and empirically interrogable.

Future Directions

Several extensions could strengthen the system: (1) a re-planning depth limit to prevent the over-expansion failure mode observed on q3; (2) embedding-based semantic deduplication of sub-questions during decomposition; (3) adversarial query generation to reduce confirmation bias; (4) human-calibrated confidence scoring; (5) hierarchical summarization to manage context window pressure; and (6) support for multi-modal evidence in the synthesis pipeline.

References

[1] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. "ReAct: Synergizing Reasoning and Acting in Language Models." *International Conference on Learning Representations (ICLR)*, 2023. https://arxiv.org/abs/2210.03629

[2] Wang, L., Xu, W., Lan, Y., Hu, Z., Lan, Y., Lee, R.K.-W., and Lim, E.-P. "Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models." *Association for Computational Linguistics (ACL)*, 2023. https://arxiv.org/abs/2305.04091

[3] Shinn, N., Cassano, F., Gopinath, A., Shukla, K., and Narasimhan, K. "Reflexion: Language Agents with Verbal Reinforcement Learning." *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. https://arxiv.org/abs/2303.11366

[4] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., and Scialom, T. "Toolformer: Language Models Can Teach Themselves to Use Tools." *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. https://arxiv.org/abs/2302.04761

[5] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., and Zhou, D. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. https://arxiv.org/abs/2201.11903

[6] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., and Polosukhin, I. "Attention Is All You Need." *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. https://arxiv.org/abs/1706.03762

[7] Erol, K., Hendler, J., and Nau, D.S. "HTN Planning: Complexity and Expressivity." *AAAI Conference on Artificial Intelligence*, 1994. https://www.cs.umd.edu/~nau/papers/erol1994htn.pdf

[8] Fox, M., Gerevini, A., Long, D., and Serina, I. "Plan Stability: Replanning versus Plan Repair." *International Conference on Automated Planning and Scheduling (ICAPS)*, 2006. https://strathprints.strath.ac.uk/2776/